

Basic (your first real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
import requests

response = requests.get("https://api.github.com")
print(response.status_code)
print(response.json())

try:
    requests.get("http://broken")
except Exception as e:
    print("Caught it:", type(e).__name__)
```

Expected output:

```
200
{'current_user_url': 'https://api.github.com/user', ...}
Caught it: MissingSchema
```

This works. It catches `Exception` broadly rather than the specific `requests` exception class, which is fine for a first pass, but it would also swallow completely unrelated bugs in the same block if you added more code later.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — the specific exception, wrapped in `main()`

```
import requests

def main() -> None:
    response = requests.get("https://api.github.com")
    print(f"status: {response.status_code}")
    print(f"body: {response.json()}")

    try:
        requests.get("http://broken")
    except requests.exceptions.RequestException as e:
        print(f"Caught a client-side request error: {type(e).__name__} - {e}")

if __name__ == "__main__":
    main()
```

Expected output:

```
status: 200
body: {'current_user_url': 'https://api.github.com/user', ...}
Caught a client-side request error: MissingSchema - Invalid URL 'http://broken': N
```

Difference from Basic: catching `requests.exceptions.RequestException` specifically (instead of bare `Exception`) means this block only swallows problems that are actually about making the request — a bug anywhere else in a bigger `try` block would still surface normally, instead of being silently caught here too. Wrapping the script in `main()` also matches the pattern from Doc01 — small habit, but it's the one you'll want once this script grows past a few lines.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — `raise_for_status()` , and telling 3 real failures apart

```
import requests

def try_call(url: str) -> None:
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
    except requests.exceptions.RequestException as e:
        print(f"{url} -> FAILED: {type(e).__name__}: {e}")
    else:
        print(f"{url} -> OK: {response.status_code}")

def main() -> None:
    urls = [
        "https://api.github.com",
        "https://api.github.com/this-path-does-not-exist-at-all",
        "https://this-domain-genuinely-does-not-exist.invalid",
        "http://broken",
    ]
    for url in urls:
        try_call(url)

if __name__ == "__main__":
    main()
```

Expected output:

```
https://api.github.com -> OK: 200
https://api.github.com/this-path-does-not-exist-at-all -> FAILED: HTTPError: 404
https://this-domain-genuinely-does-not-exist.invalid -> FAILED: ConnectionError:
http://broken -> FAILED: MissingSchema: Invalid URL 'http://broken': No scheme supp
```

Four calls, four different outcomes, and 3 different exception class names — `HTTPError` (a "successful" call that came back 404), `ConnectionError` (DNS never resolved), and `MissingSchema` (never even tried to connect). All 3 land in the same `except requests.exceptions.RequestException` block because they're all subclasses of it, but `type(e).__name__` shows they're not the same problem at all.

Difference from Intermediate: Intermediate catches one specific exception class for one specific kind of broken URL. This approach adds `raise_for_status()` so a 4xx/5xx *response* (not an exception on its own) gets folded into the same `except` path as a connection failure, and it runs several genuinely different failure causes through the same code to show that "request failed" is really an umbrella over multiple distinct problems — each of which, in a real client, would get a different response (retry a 5xx, don't retry a 404, fix the URL for a `MissingSchema`).

Which one should you actually write? For quick, throwaway exploration of an API, Intermediate's plain `try/except RequestException` is enough. The moment this code is going to run unattended (a script, a scheduled job, part of a bigger app), add `raise_for_status()` — without it, a 404 or a 500 silently looks like success to the rest of your code, since no exception was raised and `response.json()` might even still return something. This exact `raise_for_status()` habit carries straight into this document's Build Task, where classifying a status code correctly (retry it, or fail immediately) is the entire point of `http_client.py`.